Machine learning architectures

TIMELINE

I Architectures, 1943–1997 2
II The quiet years 3
III Architectures, 2012–2022 4

ARCHITECTURES

IV A feed-forward network 5
V A convolutional network 6
VI One convolution position 6
VII A recurrent network 7
VIII An autoencoder 7
IX An embedding lookup 8
X One attention head, by hand 8
XI One encoder block 9

Architectures, 1943–1997

▭ ⬭ white owns weights, grey does not

— what this row added

IDEA a new mechanism

APPLICATION existing ideas, more data

ERA DOMAIN KIND

1943 - 1969 a unit and its limit

1943 NEUROPHYSIOLOGY IDEA

McCulloch–Pitts

A set number of inputs must fire at once.

Inputs are counted, not weighted; one inhibitory input blocks the output. Count and wiring are set by hand, never fitted.



McCulloch & Pitts 1943 · doi:10.1007/BF02478259

1958 PERCEPTION IDEA

Rosenblatt

The rule adjusts weights after each wrong answer.

A weight moves by the error times that weight's input.
Settles only if one straight cut splits both groups.



Rosenblatt 1958 · doi:10.1037/h0042519

1960 SIGNAL PROCESSING IDEA

LMS, ADALINE

Each weight steps downhill on the squared error.

Each weight moves by the error times its own input.
Needs a known target for every training input.



Widrow & Hoff 1960 · IRE WESCON Conv. Rec., pt 4

1969 MATHEMATICS IDEA

Minsky & Papert

No single flat cut separates XOR’s four cases.

Two cuts suffice, so a second layer of units would do.
Nothing in 1969 could set that second layer’s weights.
Parity of n bits needs one unit wired to all n inputs.



Minsky & Papert 1969 · MIT Press

1980s - 1997 structure and credit

1980s VISION IDEA

Neocognitron → LeNet

One weight set is reused at every position.

Each small patch is multiplied by the same weights.
One filter detects one pattern, so a layer needs many.



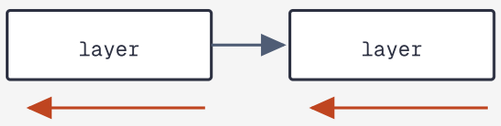
LeCun et al. 1989 · doi:10.1162/neco.1989.1.4.541

1986 COGNITIVE SCIENCE IDEA

Backpropagation

Computes the error's derivative for every weight.

Each layer: local derivative × weighted sum from above.
Cost: one backward pass, roughly one forward pass.



Rumelhart et al. 1986 · doi:10.1038/323533a0

1997 SEQUENCE IDEA

LSTM

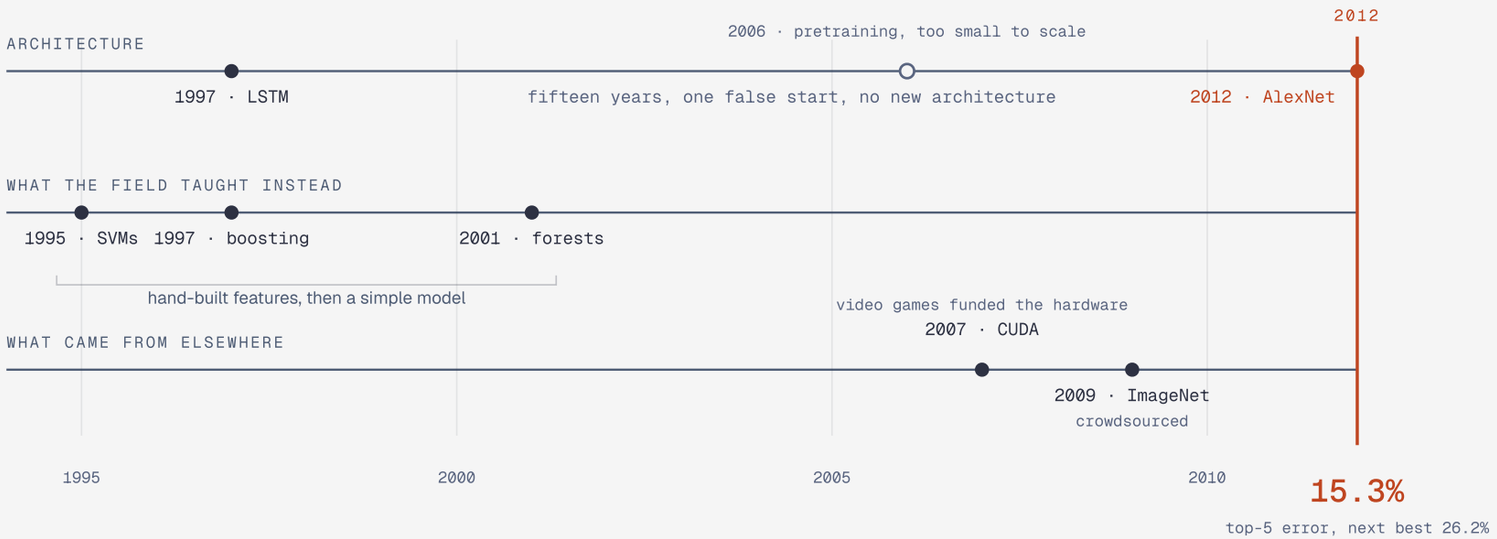
The cell state is added to, never overwritten.

Gradient flows through sums, not a product of matrices.
Cost: four times the parameters of a plain cell.



Hochreiter & Schmidhuber 1997 · doi:10.1162/neco.1997.9.8.1735

The quiet years



The three pieces came from three fields, and all three existed by 2009. 2012 added no idea; it put all three on one problem.

Architectures, 2012–2022

white owns weights, grey does not — what this row added

IDEA a new mechanism — APPLICATION existing ideas, more data

ERA · DOMAIN · KIND

2012 – 2015 applied across domains

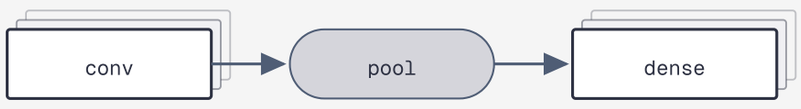
2012 · VISION · APPLICATION

AlexNet

Trains the same layers on far more data and compute.

No new mathematics; dropout and ReLU were borrowed.
60M weights, five days on two 3GB GPUs.

Krizhevsky et al. 2012 · NeurIPS 2012, vol. 25



2013 · LANGUAGE · IDEA

word2vec

Fits each word's vector to predict nearby words.

Trained so words in the same places get similar vectors.
One row per distinct word, shared by every use.

Mikolov et al. 2013 · arXiv:1301.3781



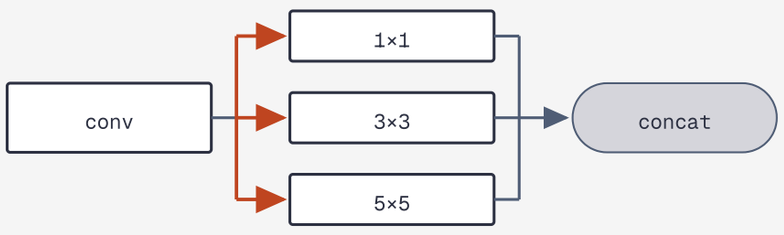
2014 · VISION · IDEA

Inception

Runs four branches on one input, then joins them.

Branches: 1×1, 3×3, 5×5 filters and a 3×3 pool.
Nothing picks a size; all four outputs are passed on.

Szegedy et al. 2014 · arXiv:1409.4842



2014 · TRANSLATION · IDEA

seq2seq + attention

Computes a new input summary per output word.

Scores every encoder state; softmax; then a weighted sum.
Cost: every output word rescores every encoder state.

Bahdanau et al. 2014 · arXiv:1409.0473



2015 · VISION · IDEA

ResNet

Each block learns a correction to its input.

A shortcut wire passes the input around the block.
Cost: one addition, no extra weights to train.

He et al. 2015 · arXiv:1512.03385



2017 – 2022 the same block, more data and compute

2017 · LANGUAGE · IDEA

Transformer

Each position compares against every position.

Pair scores set the mix weight of each position.
Doubling the input length quadruples work and memory.

Vaswani et al. 2017 · arXiv:1706.03762



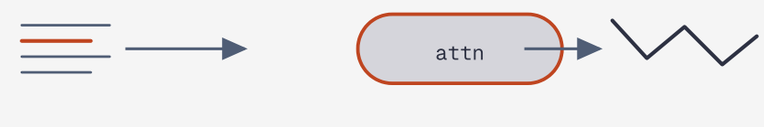
2020 · STRUCTURAL BIOLOGY · APPLICATION

AlphaFold 2

Attention over an alignment predicts the fold.

The input is an alignment: one protein, many relatives.
Same attention as 2017; the input is what changed.
CASP14: matched experimental accuracy in most cases.

Jumper et al. 2021 · doi:10.1038/s41586-021-03819-2



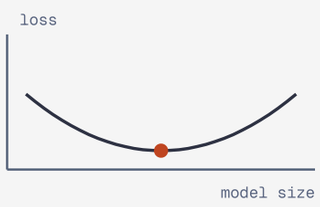
2022 · PRETRAINING · IDEA

Compute budget

Split a fixed compute budget between size and data.

Too big a model on too few tokens wastes the budget.
The best split is far smaller than the largest model that fits.

Hoffmann et al. 2022 · arXiv:2203.15556



one budget has one best model size
too small wastes tokens, too large undertrains

2022 · HUMAN PREFERENCE · IDEA

Preference tuning

Tune a model toward answers people ranked higher.

A reward model learns from answer pairs people ranked.
Cost: many raters; tuning exploits reward-model errors.

Ouyang et al. 2022 · arXiv:2203.02155



OPEN AFTER 2022 — no single mechanism orders these yet

long context · tool use and agents · test-time compute · multimodal input

A feed-forward network



MODEL d = 512 features · h = 256 hidden · K = 10 classes · B = batch

TERM	EQUATION
Linear	$z = xW + b$
ReLU	$h = \max(0, z)$
Output	$\hat{y} = \text{softmax}(z_L)$

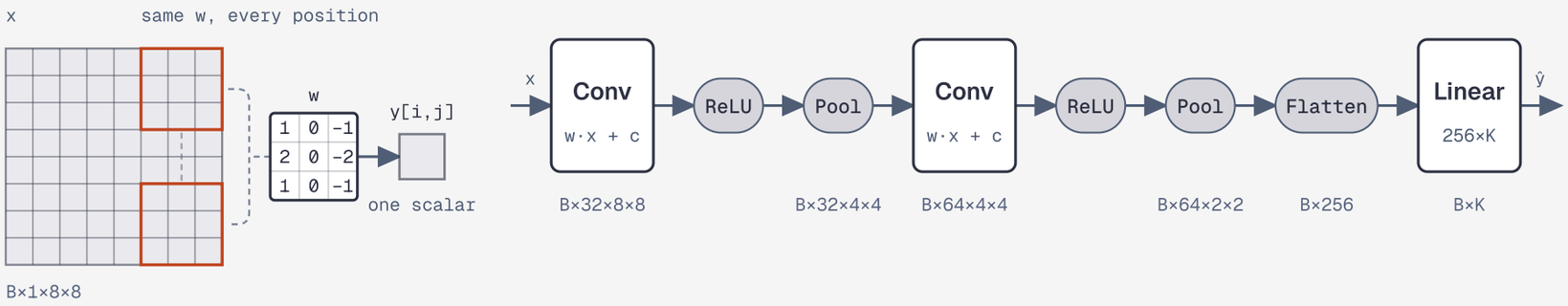
PYTORCH

```
nn.Sequential(              # the chain, left to right
    nn.Linear(512, 256),    # Linear · 512 × 256
    nn.ReLU(),              # ReLU capsule
    nn.Linear(256, 256),    # Linear · 256 × 256
    nn.ReLU(),              # ReLU capsule
    nn.Linear(256, 10))     # Linear · 256 × 10
ŷ = softmax(z_L)            # output, not a layer
```

SYMBOLS

x	input features, B × d, unordered	W	weights of one Linear	b	bias, one per output
z	pre-activation, xW + b	ReLU	max(0, z), elementwise	h	hidden width, 256
softmax	over K scores; each row sums to 1	ŷ	class probabilities, B × K	z_L	pre-activation of the last Linear
B	examples per batch	d	input features, 512	K	classes, 10

A convolutional network



MODEL C = 1 channel · H = W = 8 · k = 3 kernel · K = 10 classes · B = batch

TERM	EQUATION
Conv	$y[i,j] = \Sigma_{a,b}\ w[a,b] \cdot x[i+a,\ j+b] + c$
ReLU	$\mathrm{ReLU}(z) = \max(0, z)$
Pool	max over a 2×2 window, stride 2
Flatten	B×64×2×2 → B×256

PYTORCH

```
nn.Conv2d(1, 32, kernel_size=3, padding=1)   # k=3, pad 1
nn.MaxPool2d(2)                              # 4×4
nn.Conv2d(32, 64, kernel_size=3, padding=1)  # 32→64
nn.MaxPool2d(2)                              # 2×2
nn.Flatten()                                 # 64·2·2=256
nn.Linear(256, 10)                           # 256→K
```

SYMBOLS

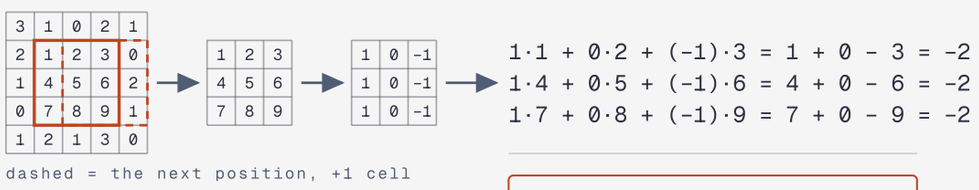
x	one input channel, H×W	y	one element of the conv output	ŷ	class scores, B×K
w	one k×k filter, reused at all positions	i, j	row, column of that element	32, 64	filters in each conv layer
c	bias, one per filter	a, b	row, column inside the k×k window	256	flattened length, 64·2·2
z	conv output before ReLU	w·x	dot product over one window	2×2	pool window, stride 2

One convolution position

$$y = \Sigma_k\ w_k\ x_k$$

SYMBOL	NAME	WHAT IT IS
y	output	number; the value written for this one window position
Σ_k	sum	operator; add the nine terms, one per cell of the window
k	cell index	index; 1 to 9, which cell of the window
w_k	filter weight	number; the filter weight at cell k, the same nine at every position
x_k	pixel	number; the image value at cell k inside this window

WORKED · IMAGE 5×5 · 1 CHANNEL · WINDOW 3×3 · FILTER 3×3 · STRIDE 1 · NO PADDING



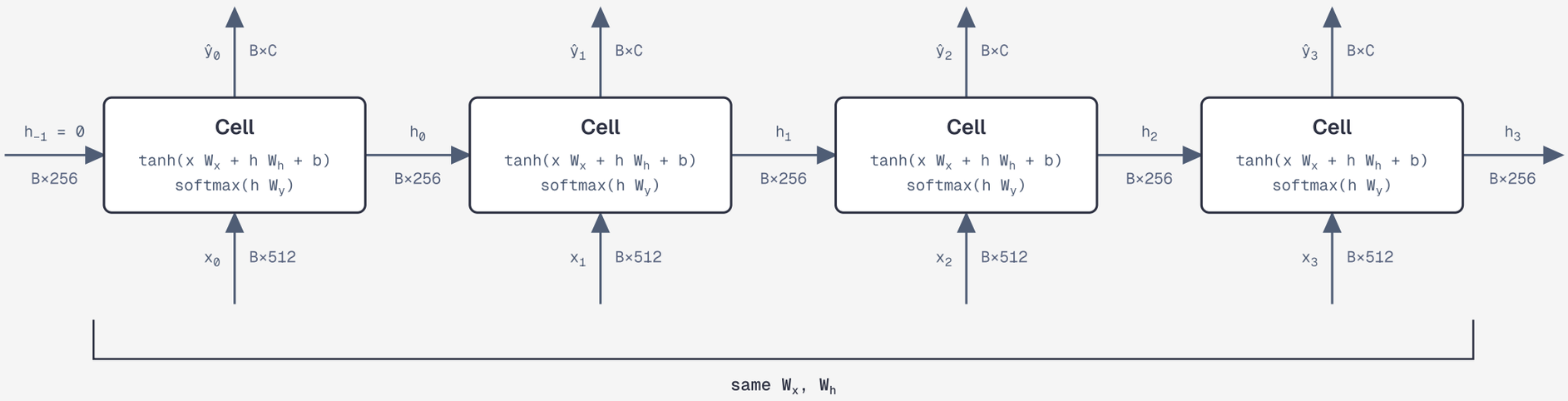
NINE PRODUCTS, ONE ROW AT A TIME

1·1 + 0·2 + (−1)·3 = 1 + 0 − 3 = −2
1·4 + 0·5 + (−1)·6 = 4 + 0 − 6 = −2
1·7 + 0·8 + (−1)·9 = 7 + 0 − 9 = −2

y = −2 − 2 − 2 = **−6**

at the border the window runs off the image: with no padding those x_k are missing, so y is undefined

A recurrent network



MODEL d = 512 input · h = 256 state · T = 4 steps · B = batch

TERM | EQUATION

Cell	$h_t = \tanh(x_t\ W_x + h_{t-1}\ W_h + b)$
Output	$\hat{y}_t = \mathrm{softmax}(h_t\ W_y)$
Init	$h_{-1} = 0$

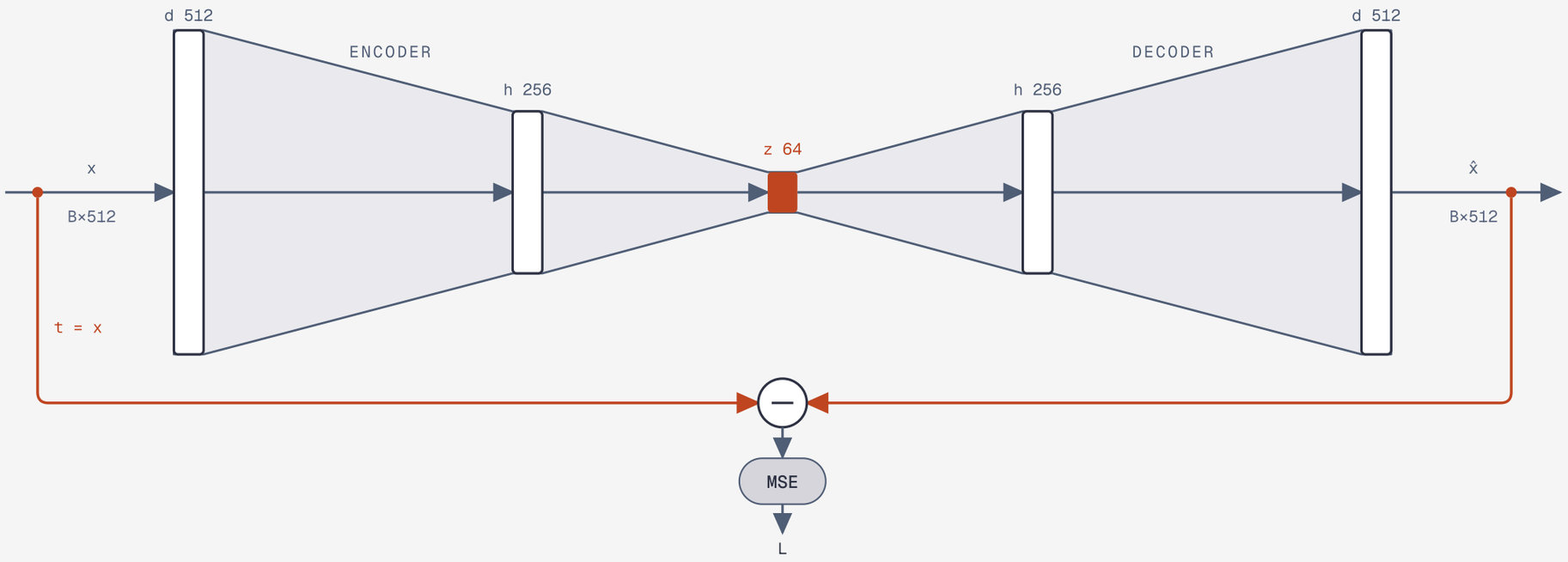
PYTORCH

```
nn.RNN(input_size=512,       # d
       hidden_size=256,      # h
       batch_first=True)     # x: B×T×512
# defaults: nonlinearity='tanh', bias=True
nn.Linear(256, C)            # Wy
```

SYMBOLS

x_t	input at step t, 512 numbers	W_x	512 × 256 input weights	tanh	elementwise; output in (−1, 1)
h_t	state out of step t, 256 numbers	W_h	256 × 256 state weights	softmax	over C; each row sums to 1
h_{-1}	zero state entering step 0	W_y	256 × C readout weights	C	number of output classes
$\hat{y}_t$	distribution over C classes	b	bias, length 256	t	step index

An autoencoder



MODEL d = 512 input · h = 256 hidden · z = 64 code · B = batch

TERM | EQUATION

Encoder	$z = \mathrm{ReLU}(\mathrm{ReLU}(x\ W_1{+}b_1)\ W_2{+}b_2)$
Decoder	$\hat{x} = \mathrm{ReLU}(\mathrm{ReLU}(z\ W_3{+}b_3)\ W_4{+}b_4)$
Loss	$L = \mathrm{mean}((x - \hat{x})^2)$
Target	t = x · the input, not a label

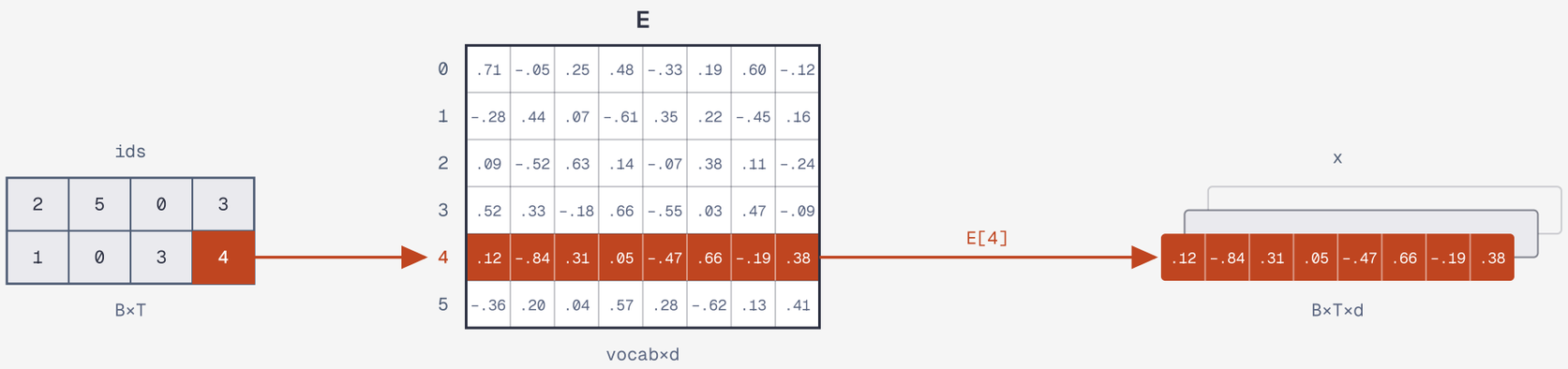
PYTORCH

```
enc = nn.Sequential(                  # x → z
    nn.Linear(512, 256), nn.ReLU(),   # d → h
    nn.Linear(256, 64))               # h → z
dec = nn.Sequential(                  # z → x̂
    nn.Linear(64, 256), nn.ReLU(),    # z → h
    nn.Linear(256, 512))              # h → d
loss = F.mse_loss(x_hat, x)           # x̂ vs x
```

SYMBOLS

x, x̂	input and its reconstruction	mean	average over the batch	t	the target; t is the input x
W_1–W_4	512×256, 256×64, 64×256, 256×512	⊖	x − x̂, the residual	bar	a layer; height = its width
b_1–b_4	one bias vector per Linear	MSE	mean of the squared residual	wedge	one learned map, e.g. W_1
ReLU	max(0, a)	L	the loss, one number	accent	the path carrying x to the loss

An embedding lookup



MODEL vocab = 6 · d = 8 · B = batch · T = positions

TERM	EQUATION
Index	ids: integers in 0..vocab-1
Lookup	x = E[ids]
Shape	B×T → B×T×d
Table	E: vocab×d, learned

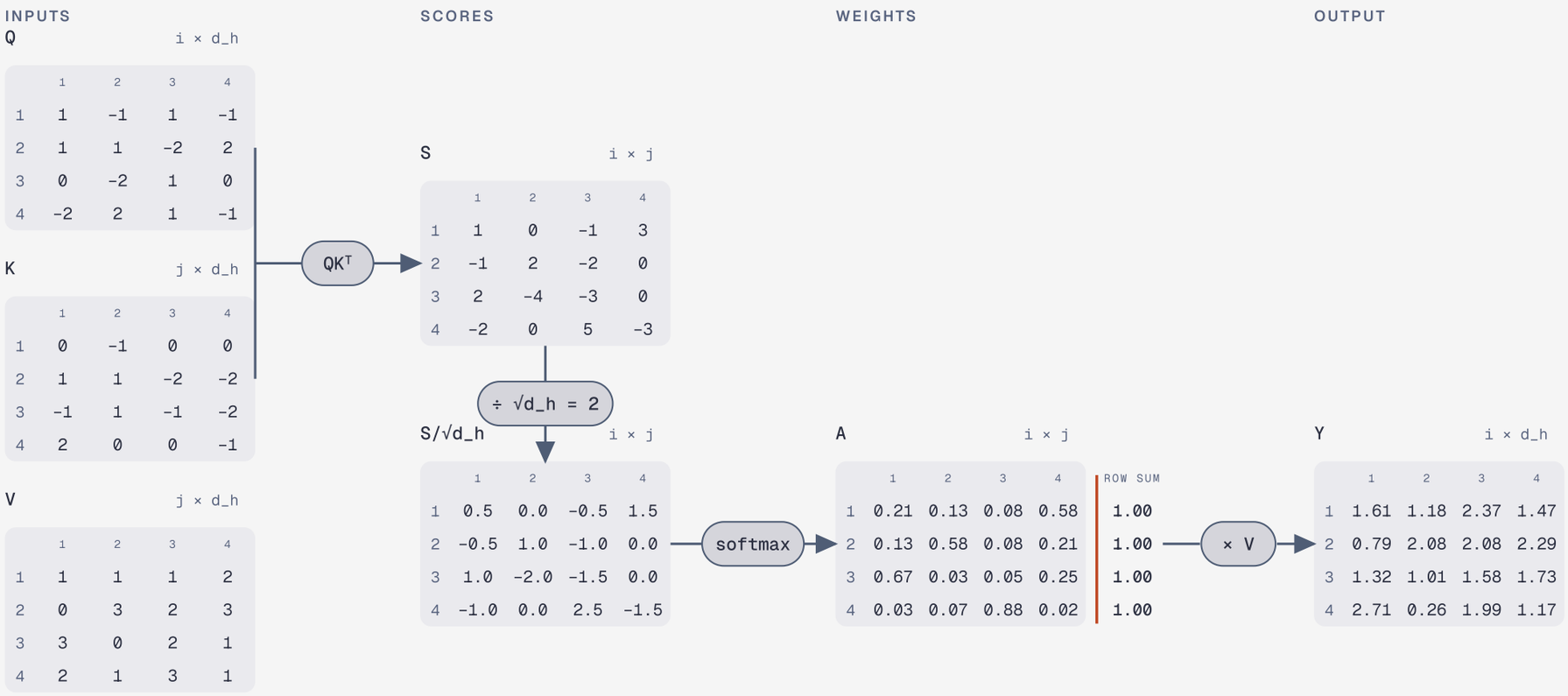
PYTORCH

```
emb = nn.Embedding(
    num_embeddings=vocab,   # rows of E
    embedding_dim=d)        # width of a row
emb.weight                  # E: vocab×d, learned
x = emb(ids)                # E[ids], one row each
                            # x: B×T×d
```

SYMBOLS

ids	integer token ids, one per position	vocab	how many ids exist; rows of E	0..5	the ids; each names one row
E	the embedding table; one row per id	d	numbers in each row	B×T×d	one d-vector per position
x	vectors taken out of E	B	sequences in a batch		
E[4]	row 4 of E; d numbers, same order	T	positions per sequence		

One attention head, by hand



MODEL T = 4 positions · d_h = 4 head width (= d/h) · one head · B = 1

TERM	EQUATION
scores	$S = QK^T$
weights	A = softmax over j of (S / √d_h)
output	Y = A V

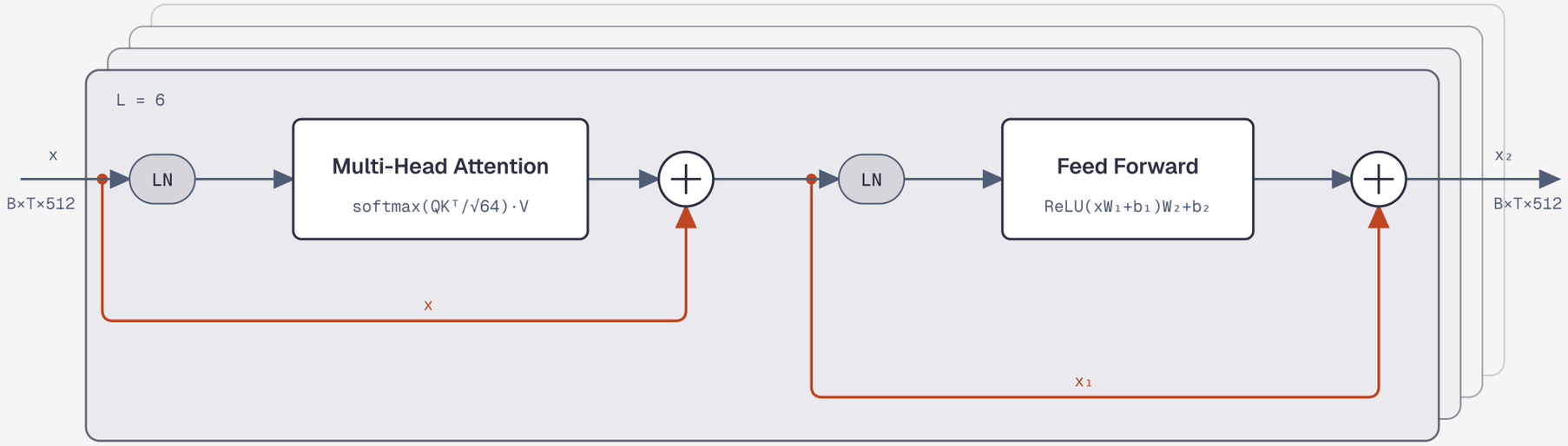
PYTORCH

```
F.scaled_dot_product_attention(   # (B, 1, T, d_h)
q, k, v,                          # S = q kᵀ / √d_h
scale=1/2)                        # √d_h = √4 = 2
                                  # A = softmax over j
                                  # Y = A v
```

SYMBOLS

Q	query; x W_Q; what i looks for	S	similarity scores, before scaling	Y	output A V; row i mixes all V
K	key; x W_K; what j offers	√d_h	scale; √4 = 2	1.00	row sum of A, rounded to 2 decimals
V	value; x W_V; what j passes on	softmax	over j; each row sums to 1	d_h	head width; 4 features per head
QK^T	one dot product per (i, j) pair	A	weights; row i mixes all keys	i, j	position index; 1 to 4 here

One encoder block



MODEL d = 512 features · h = 8 heads · d/h = 64 · B = batch · T = positions

TERM	EQUATION
LN	$LN(x) = (x - \mu)/\sigma \cdot \gamma + \beta$
MHA	$MHA(x) = softmax(QK^T/\sqrt{64})\cdot V$
FFN	$FFN(x) = ReLU(xW_1+b_1)W_2+b_2$
⊕	$x + F(LN(x))$

PYTORCH

```
nn.TransformerEncoderLayer(
d_model=512, # d
nhead=8, # h
dim_feedforward=2048, # 4d
activation=nn.ReLU(), # max(0, z)
norm_first=True) # LN inside
```

SYMBOLS

μ, σ	mean, std of one row's d features	ReLU	max(0, z)	F	sublayer inside the branch
γ, β	learned scale and shift, length d	W_1, W_2	d × 4d, 4d × d	⊕	elementwise addition
Q, K, V	xW_Q, xW_K, xW_V	b_1, b_2	bias vectors	x, x_1, x_2	stream before, between, after
softmax	over j; each row sums to 1	√(d/h)	= √64 = 8		